SPRAWOZDANIE NR 4			
Nazwa ćwiczenia	MLFlow, wersjonowanie modelu, logowanie hiperparametrów i metryk		POLITECHNIKA BYDGOSKA Wydział Telekomunikacji Informatyki i Elektrotech
Przedmiot	Nowoczesne Technologie Przetwarzania Danych		
Student grupa	Mikołaj Domagała gr. 2		
Data ćwiczeń	26.03.2026	02.04.2026	Data oddania sprawozdania

Cel ćwiczenia :

Celem ćwiczenia jest tworzenie obrazu Dockera z modelem ML; uruchamianie kontenera i testowanie endpointów; konfiguracja Docker Compose dla aplikacji ML.

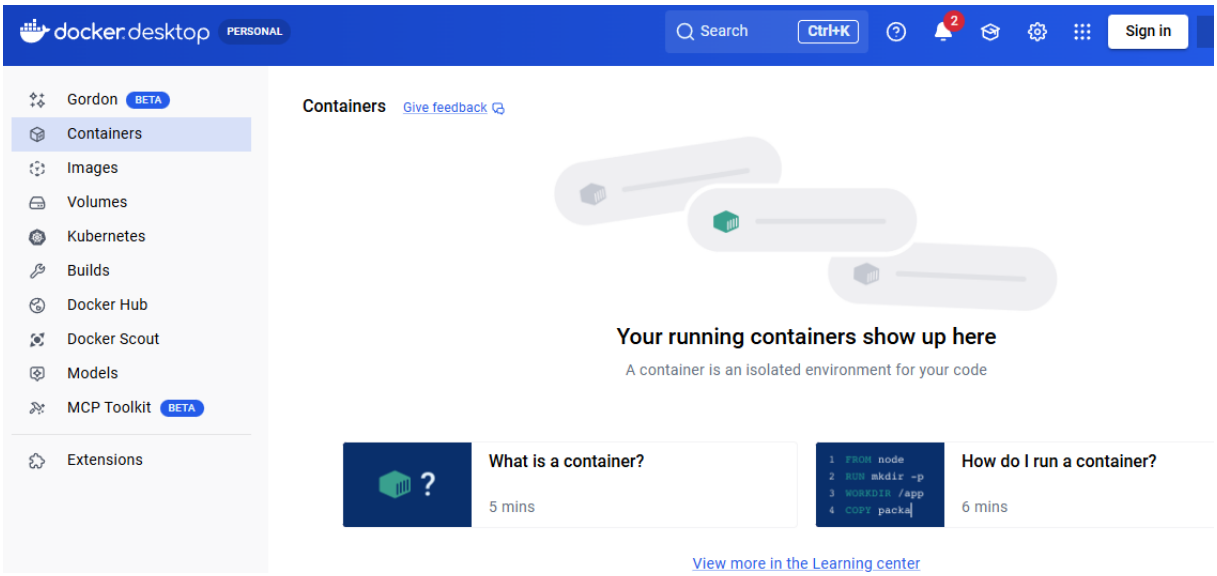
Przebieg :

Stworzyłem plik requirements.txt z potrzebnymi bibliotekami

```
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD> pip freeze > requirements.txt
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD>
```

```
absl-py==2.4.0
astunparse==1.6.3
attrs==25.4.0
certifi==2026.2.25
charset-normalizer==3.4.5
colorama==0.4.6
contourpy==1.3.3
cycler==0.12.1
dm-tree==0.1.9
docstring_parser==0.17.0
einops==0.8.2
etils==1.14.0
flatbuffers==25.12.19
fonttools==4.61.1
fsspec==2026.2.0
gast==0.7.0
google-pasta==0.2.0
googleapis-common-protos==1.73.0
grpcio==1.78.0
```

Następnie zainstalowałem dockera



Stworzyłem dockerfile z wersją slim aby obraz mniej ważył

```
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["uvicorn", "NTPD3:app", "--host", "0.0.0.0", "--port", "8080"]
```

Następnie w terminalu uruchomiłem komendę która buduje dockera

```
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD> docker build -t ntpd-app .
[+] Building 390.5s (10/10) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                     0.0s
 => => transferring dockerfile: 214B                                                     0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                      1.3s
```

Uruchomiłem kontener oraz za pomocą docker ps sprawdziłem czy obraz się zbudował, status potwierdził że kontener wstał

```
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD> docker run -d -p 8080:8080 --name ntpd-container ntpd-app
5dfa4035cff2ebe91b72fcadded112c2756ba1d729cee4d9513e47ff175ff9a8
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS
                     NAMES
5dfa4035cff2   ntpd-app   "uvicorn NTPD3:app -…"   5 seconds ago   Up 4 seconds   0.0.0.0:8080->8080/tcp, [
::]:8080->8080/tcp   ntpd-container
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD>
```

Do testowania poprawności dockerfile i modelu ml dodałem data json ze zmienną hours 10

```
{"hours": 10}
```

A następnie przetestowałem zapytanie za pomocą curl

```
(.venv) PS C:\Users\niedo\PycharmProjects\NTPD> curl.exe -X POST "http://localhost:8080/predict"
>>      -H "Content-Type: application/json" `
>>      --data "@data.json"
{"input_hours":10.0,"predicted_score":102.0}
```

Curl wysłał output z poprawną ilością godzin oraz ze swoją predykcją

Do testowania drugiego serwisu bazy danych Redis dodałem w moim głównym pliku z api testowanie z wypisaniem czy to działa

```
cache = redis.Redis(host='redis-db', port=6379, decode_responses=True)
@app.get("/redis-test")
def test_redis():
    try:
        cache.set("test_key", "polaczenie dziala")
        value = cache.get("test_key")
        return {"redis_status": "success", "data": value}
    except Exception as e:
        return {"redis_status": "error", "message": str(e)}
```

Oraz stworzyłem plik docker-compose który uruchamia ml oraz ten drugi serwis

```yaml
version: '3.8'
services:

  ml-app:
    build: .
    container_name: ntpd-ml-container
    ports:
      - "8080:8080"
    networks:
      - ml-network
    depends_on:
      - redis-db

  redis-db:
    image: "redis:alpine"
    container_name: redis-cache
    ports:
      - "6379:6379" #do bazy danych redis
    networks:
      - ml-network

networks:
  ml-network:
    driver: bridge
```

Za pomocą docker-compose up -d odpaliłem kontener oraz na localhoscie w przeglądarce przetestowałem aplikację za pomocą swaggera i wykonałem zapytanie post

POST /predict Predict

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "hours": 10
}
```

Swagger zwrócił poprawną odpowiedź

200

Response body

```
{
  "input_hours": 10,
  "predicted_score": 102
}
```

Instrukcja uruchamiania rozpisana w readme :

```
1. Stwórz środowisko wirtualne

python -m venv .venv

2. aktywuj środowisko

.venv\Scripts\activate

3. zainstaluj dependencies

pip install -r requirements.txt

4. uruchom serwer

uvicorn NTPD3:app --reload --port 8080

5. testy

uruchomienie przeglądarki pod http://localhost:8080/docs
następnie można przetestować /predict i wpisanie ilości godzin np. hours : 10

W terminalu za pomocą curla :
curl.exe -X POST "http://localhost:8080/predict" -H "Content-Type: application/json" -d "{\"hours
```

Wnioski :

W tym laboratorium nauczyłem się obsługiwać dockera tworzyć jego obraz oraz uruchamiać kontenery a następnie testować czy działają